Chapitre 3:

Les patrons de conception de création

Les patrons de conception de création

- Des solutions permettant de mieux gérer la création des objets.
- Les patrons de création cherchent notamment à :
  - Contrôler la manière dont les objets sont créés
  - Réduire le couplage entre le code client et les classes concrètes
  - Faciliter l'évolution du système
  - Rendre la création des objets plus flexible

Pourquoi s'intéresser à la création des objets ?

- La création d'un objet semble simple :

```
Client client = new Client();
```

- Mais dans une application réelle, la création peut être plus complexe :

```
Client client = new Client(nom, adresse, email, ...);
```

- Ou dépendre :
  - du type d'objet à créer
  - de la configuration de l'application
  - du contexte d'utilisation
  - de paramètres nombreux ou complexes
  - d'une logique de création qui peut évoluer

- Faut-il toujours utiliser directement new ?

Le problème du couplage

- Considérons :

```
public void traiter() {
        Client client = new Client();
        // ...
    }
}
```

- L'application dépend directement de la classe **`Client`**. Le problème se pose si la manière de créer le client change.
- **Problème :**
  - La logique métier est couplée à la manière concrète de créer les objets.
- **Objectif :**
  - Séparer **l'utilisation d'un objet** de **sa création**.

Encapsuler la création

- Le code client devrait autant que possible utiliser un objet sans avoir à connaître les détails de sa création.

Sans encapsulation

Le code client crée directement l'objet.

Application.java

```java
public class Application {
    public void traiter() {
        Client client = new Client();
        // ..
    }
}
```



Conséquence

L'application est couplée à la classe concrète **`Client`**.
Si la manière de créer le client change, il faut modifier le code client.

Avec encapsulation

La création est déléguée à un composant dédié.

Application.java

```java
public class Application {
    public void traiter() {
        Client client = ClientFactory.getClient();
        // ..
    }
}
```



Avantages

- Le code client n'est plus couplé à une classe concrète.
- La création peut évoluer sans modifier le code client.
- On peut facilement changer de type d'objet, de configuration, etc.

- Cela permet de faire évoluer la création sans modifier nécessairement le code qui utilise l'objet.

Les patrons de création du GoF

- Le **Gang of Four** définit **5 patrons de création** :

Patron	Idée principale
Singleton	Garantir une instance unique
Factory Method	Déléguer la création à une méthode spécialisée
Abstract Factory	Créer des familles d'objets liés
Builder	Construire progressivement un objet complexe
Prototype	Créer un objet à partir d'un objet existant

Attention : les patrons ne remplacent pas `new`

- Les patrons de création **ne signifient pas** :
  - « Il ne faut jamais utiliser **`new`**. »
- Ils permettent plutôt de répondre à des situations où la création directe devient :
  - trop complexe
  - trop fortement couplée
  - difficile à faire évoluer
  - répétitive
  - dépendante du contexte
- **Principe :**
  - Utiliser un patron lorsqu'il apporte une réelle valeur, et non simplement parce qu'il existe.

Les patrons de conception de création

Singleton

Exemple: classe de journalisation (Logger)

- Qu'est-ce qui ne va pas avec ce code ?

```
public class Logger {

   public Logger() { } // Constructeur

   public void logMessage() {
         //Ouvre le fichier "log.txt"
         //Ecrit un message
         //ferme le fichier
   }
}
```

Exemple: classe de journalisation (Logger)

- Puisqu'il existe une ressource partagée externe («**`log.txt`**»), nous voulons contrôler étroitement la manière dont nous communiquons avec elle.

- Nous ne devrions pas avoir à instancier la classe **`Logger`** à chaque fois que nous voulons accéder à cette ressource partagée. Y a-t-il une raison de le faire ?

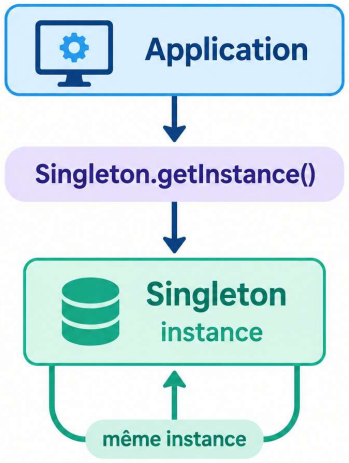
- Il nous en faut une seule.

Le problème : une seule instance nécessaire

- Idée : certaines ressources ou certains services doivent être représentés par un seul objet partagé dans l'application.
- Exemples :
  - un gestionnaire de journalisation (Logger)
  - un cache partagé
  - un registre de configuration
  - une ressource externe partagée
- Question :
  - Comment garantir qu'une classe ne possède qu'une seule instance tout en permettant d'y accéder depuis différents endroits du programme ?

Le patron Singleton

- **Définition GoF :** Le Singleton garantit qu'une classe ne possède qu'une seule instance et fournit un point d'accès global à cette instance.



- Le Singleton repose principalement sur deux mécanismes :
  - Constructeur privé: empêcher la création directe avec new
  - Méthode statique: fournir l'unique instance

Implémentation simple en Java

```
public class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```

Implémentation simple en Java

- **Utilisation :**

```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();

System.out.println(s1 == s2);
```

- **Résultat :**

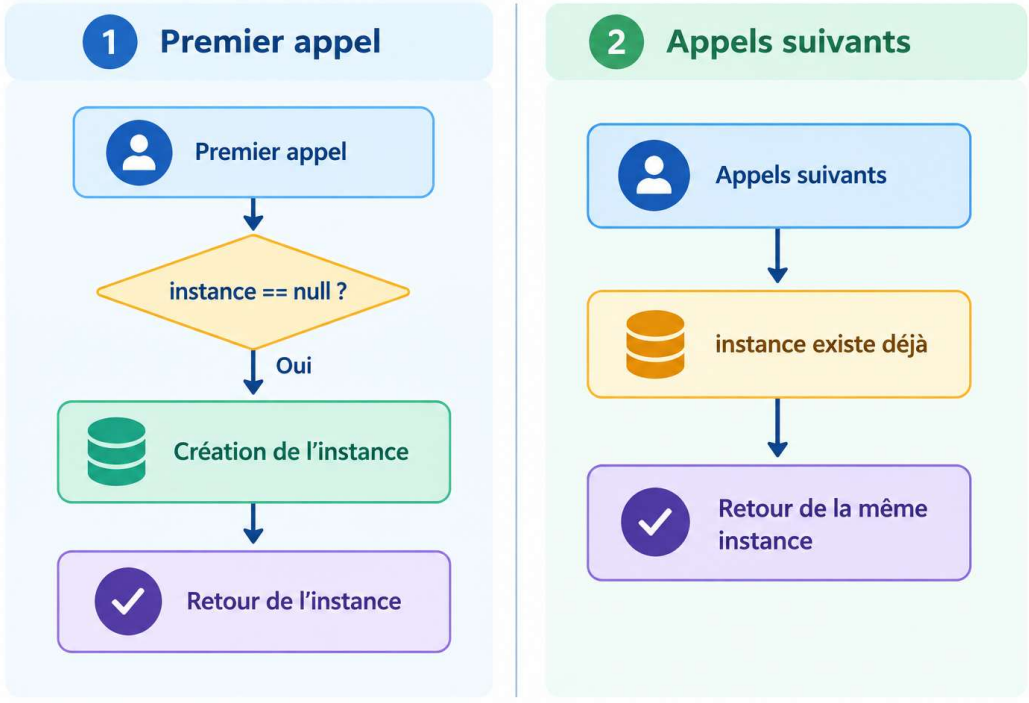
```
true
```

s1 et s2 désignent **le même objet**.

La création paresseuse (*Lazy Instantiation*)

- L'instance n'est créée **que lorsqu'elle est nécessaire.**

```
if (instance == null) {
    instance = new Singleton();
}
```



- Avantage :
  - l'objet n'est pas créé inutilement
  - la création peut être reportée jusqu'au premier besoin

La classe Logger sous forme d'un singleton

```
public class Logger {
    private Logger() { } // Constructeur
    private static Logger uniqueInstance;

    public static Logger getInstance() {
      if (uniqueInstance == null)
            uniqueInstance = new Logger();
        return uniqueInstance;
    }

    public void logMessage() {
          //Ouvre le fichier "log.txt"
          //Ecrit un message
          //ferme le fichier
    }
}
```

La classe Logger sous forme d'un singleton

- Une application peut avoir besoin d'écrire plusieurs messages dans **un même fichier de journalisation**, depuis des parties différentes de l'application:
- Sans Singleton :

```
Logger l1 = new Logger();
Logger l2 = new Logger();
Logger l3 = new Logger();
```

  - Plusieurs objets peuvent accéder à la même ressource.

- Avec Singleton :

```
Logger logger1 = Logger.getInstance();
Logger logger2 = Logger.getInstance();

logger1.log("Début du programme");
logger2.log("Traitement terminé");
```

Singleton vs. Variable statique

- Si on n'a pas créé de **`Singleton`** pour la classe **`Logger`** ?

- Imaginons que le constructeur **`Logger()`** effectue une configuration importante.
- Dans notre fichier de programme principal, on aura le code suivant :

```
public static Logger globalLogger = new Logger();
```

- Toute la configuration du **`Logger`** sera effectuée, quelque soit on a besoin d'enregistrer des logs ou non.

Singleton et Multithreading

```
public class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```

Que se passerait-il si deux threads différents accèdent à cette ligne en même temps ?

Singleton et Multithreading

- Si deux threads T1 et T2 exécutent simultanément :

```
if (instance == null) {
    instance = new Logger();
}
```

- Un scénario possible est :
  - T1 vérifie **`instance == null`** : vrai.
  - T2 vérifie **`instance == null`** : vrai.
  - T1 crée un objet **`Logger`**.
  - T2 crée un autre objet **`Logger`**.
- **Conséquence** : deux instances peuvent être créées, ce qui viole le principe du Singleton.
- **Solutions?**
  - Verrouillage simple (*Simple Locking*)
  - Verrouillage à double vérification (*Double-Checked Locking*)
  - Initialisation anticipée (*Eager initialization*)

Verrouillage simple

```
public class Singleton {
  private static Singleton instance;
  private Singleton() {}
  public static synchronized  Singleton getInstance() {
      if (instance == null) {
          instance = new Singleton();
      }
      return instance;
  }
}
```

En ajoutant le mot-clé **`synchronized`** à **`getInstance()`**, on force chaque thread à attendre son tour avant de pouvoir entrer dans la méthode. Autrement dit, deux threads ne peuvent pas entrer dans la méthode simultanément.

Verrouillage simple

- Cela résout le problème. Mais la synchronisation est coûteuse, est-ce que vraiment c'est un problème ?

- Oui, la situation est en fait un peu pire que ce qu'on pense: la synchronisation n'est utile que lors du premier passage dans cette méthode. En d'autres termes, une fois que la variable **`uniqueInstance`** a été initialisée avec une instance de **`Singleton`**, nous n'avons plus besoin de synchroniser cette méthode. Après ce premier passage, la synchronisation représente une surcharge totalement inutile.

Verrouillage à double vérification

- Avec le verrouillage à double vérification, nous commençons par vérifier si une instance a été créée ou non.
- Si ce n'est pas le cas (n’a pas été créée encore), nous procédons alors à la synchronisation.
- De cette façon, la synchronisation n'a lieu que lors du premier passage, ce qui correspond exactement à ce qu’on veut.

Verrouillage à double vérification

```
public class Singleton {
  private volatile static Singleton instance;
  private Singleton() {}
  public static Singleton getInstance() {
      if (instance == null) {
        synchronized (Singleton.class) {
          if (instance == null) {
            instance = new Singleton();
          }
        }
      }
      return instance;
  }
}
```

Vérifier la présence d'une instance. Si elle n'existe pas, entrer dans un bloc synchronisé.

On ne synchronise que lors du premier passage !

Une fois dans le bloc, vérifiez à nouveau et, si la valeur est toujours nulle, créez une instance.

Verrouillage à double vérification

- Le mot-clé **`volatile`** garantit que plusieurs threads gèrent correctement la variable **`instance`** lors de son initialisation avec l'instance **`Singleton`**.

- Le mot-clé **`volatile`** sert à garantir que les différents threads lisent la valeur la plus récente de la variable **`instance`**, conformément aux règles de visibilité mémoire de Java.

Verrouillage à double vérification

- Supposons que le thread T1 crée l'instance et que le thread T2 appelle ensuite **`getInstance()`**.
- Sans **`volatile`**, en présence d'accès concurrents, les règles de visibilité mémoire ne garantissent pas à elles seules que T2 observe la valeur écrite par T1.
- Avec volatile :
  - **Visibilité** : lorsqu'un thread écrit une nouvelle valeur dans **`instance`**, les lectures ultérieures de cette variable par les autres threads observent cette écriture selon les règles du modèle mémoire Java.
  - **Pas d'exclusion mutuelle** : **`volatile`** ne bloque pas les threads et ne garantit pas, à lui seul, qu'une seule instance sera créée.

`synchronized` et `volatile` : quelle différence ?

- **`synchronized`**: assure l'exclusion mutuelle et établit les garanties de visibilité associées au *verrou*.
- **`volatile`**: assure la visibilité des lectures et écritures de la variable entre les threads.
- Dans l'implémentation de verrouillage à double vérification:
  - La première vérification évite de prendre le *verrou* lorsque l'instance existe déjà.
  - La deuxième vérification empêche deux threads de créer chacun une instance.
  - **`volatile`** garantit la visibilité et empêche notamment qu'une référence soit observée avant que l'initialisation de l'objet soit correctement publiée.

Initialisation anticipée

- Optez pour une instance créée de manière anticipée plutôt que pour une instance créée à la demande (création paresseuse: *lazy Instantiation*).

- Si l'application crée et utilise systématiquement une instance du **`Singleton`**, ou si le surcoût lié à sa création et à son exécution n'est pas excessif, il peut être préférable de créer le **`Singleton`** de manière anticipée.

Initialisation anticipée

```
public class Singleton {

    private static Singleton instance = new Singleton();

    private Singleton() {
    }

    public static Singleton getInstance() {
        return instance;
    }

    // ...
}
```

Créer une instance de Singleton dans un initialiseur statique. L'environnement d'exécution garantit que ceci est sûr pour les threads.

On dispose déjà d'une instance, il suffit donc de la renvoyer.

Singleton et l'héritage

```
public class BaseSingleton {
   private static BaseSingleton instance;
   protected int someInt;

   private BaseSingleton() { }

   public static BaseSingleton getInstance() {
        if (instance == null) {
                instance = new BaseSingleton();
        }
        return instance;
   }

   public void doSomething() {
        someInt = 1;
   }
}
```

Singleton et l'héritage

```
public class SubClassSingleton extends BaseSingleton {
   private SubClassSingleton() { }

   @Override
   public void doSomething() {
        someInt = 2;
   }

   public void newFunction() {
        //new functionality here
   }
}
```

Question n° 1 : Quel est le problème avec le constructeur de la sous-classe **`SubClassSingleton`** ?

Singleton et l'héritage

- Voici le code qui appelle le Singleton :

```
public class Main {

    public static void doStuff() {

01    BaseSingleton.getInstance().doSomething();

02    SubClassSingleton.getInstance().doSomething();

03    SubClassSingleton.getInstance().newFunction();

    }
}
```

Singleton et l'heritage: questions

- **Question n° 2** : Pour la ligne 01, quelle est la valeur de `someInt` après l'appel ?

- **Question n° 3** : Pour la ligne 02, quelle est la valeur de `someInt` après l'appel ?

- **Question n° 4** : Quel est le problème avec la ligne 03 ?

Singleton et l'heritage: réponses

- Question n° 1: La compilation échouera. Le constructeur de la classe de base doit passer de **`private`** à **`protected`**.

- Question n° 2: **`1`**

- Question n° 3: **`1`**
  - Même si nous avons redéfini la classe de base, cela n'a pas d'importance. C'est l'implémentation de base qui est renvoyée par **`getInstance()`**.

- Question n° 4 – La compilation échouera !

Singleton et l’heritage: registre de singletons

- Le « Gang of Four » recommande l'utilisation d'un registre de Singletons. La classe de base lit une variable pour déterminer quel Singleton utiliser.

Singleton et l'heritage: registre de singletons

```
public class BaseSingleton {
  protected BaseSingleton() { }
  private static HashMap map = new HashMap();
  public static BaseSingleton getInstance(String className){
   // Récupérer le Singleton depuis la HashMap
   BaseSingleton singleton = (BaseSingleton)map.get(className);
   if (singleton != null)
     return singleton;
   else {     // Singleton non trouvé
       if (className.equals("SubClassSingleton"))
             singleton = new SubClassSingleton();
       else
             // ......
       // Ajoute le singleton à la HashMap afin de pouvoir le
             récupérer ultérieurement.
       map.put(className, singleton);
       return singleton;
}
```